

A proposal for modular macros

Document number: P0877R0

Audience: EWG

Author: [Bruno Cardoso Lopes](#)

11 February 2018

Motivation & Background

An overall description of Apple's software ecosystem and its association with modules was discussed in Albuquerque's paper [P0841R0](#).

This paper continues that discussion but focuses specifically on addressing the support for macros. Apple is not at all special in this regard; the whole C++ library ecosystem depends on vending preprocessor macros.

Where Apple is special is in having experience deploying modules at scale across existing mixed C++, C, Objective C, and Objective C++ codebases, of its own and in its ecosystem. We've been through a feedback loop after trying to limit/ban macros, which proved that's too onerous for users to migrate to a C++ world without macro support.

Using the macOS SDK as an example we identified a number of use-cases that are not supported by the Modules TS. Here is a proposal to augment the TS to support these cases.

Macros in macOS SDK

Apple's library interfaces support different C-based languages, macros are used in order to correctly gather availability information, heterogeneous platform support and to reason on top of features. The macOS SDK is completely modularized and macros are available for consumption at any library interface level, being critical in Apple's chain of module imports.

For example, as one can see in `LinearAlgebra/base.h` macOS 10.13 SDK, the header use macros to control availability information for a library:

```
/* Define abstractions for a number of attributes that we wish to be able to
   concisely attach to functions in the LinearAlgebra library. */
#define LA_AVAILABILITY __OSX_AVAILABLE_STARTING(__MAC_10_10, __IPHONE_8_0)
...
#define LA_FUNCTION      OS_EXPORT OS_NOTHROW
#define LA_CONST         OS_CONST
```

Note that `__MAC_10_10` is available through another module that provides macros from `Availability.h`. Another example is the `Foundation` framework, which has been part of Apple's ecosystem for decades and define macros that are used in almost all other frameworks in the SDK. Example:

```
...
#define NS_AVAILABLE(_mac, _ios) CF_AVAILABLE(_mac, _ios)
#define NS_AVAILABLE_MAC(_mac) CF_AVAILABLE_MAC(_mac)
#define NS_AVAILABLE_IOS(_ios) CF_AVAILABLE_IOS(_ios)

...
#ifndef NS_ASSUME_NONNULL_BEGIN
#define NS_ASSUME_NONNULL_BEGIN _Pragma("clang assume_nonnull begin")
#endif
```

For instance, `NS_ASSUME_NONNULL_BEGIN` and `NS_AVAILABLE` are used 16297 and 37929 times respectively by other framework headers in the SDK. Note that `NS_AVAILABLE` is also defined in terms of the macro `CF_AVAILABLE`, which is defined in the CoreFoundation framework. The same pattern repeats for hundreds of other macros in several other frameworks.

Additionally, one might argue that a user could import a module for Foundation and still `#include` the header to have the macro functionality available. However, users are encouraged to use a framework by including its [umbrella header](#), e.g., `#include <Foundation/Foundation.h>`, and not to directly include other headers from the framework. It seems odd that the user, after importing from Foundation would also need to `#include` the umbrella header to get such macros; there wouldn't be any compile time benefit and work is done twice.

Usage in Open Source

The usage of such macros isn't limited to Apple headers. For instance, [WebKit](#) is an open source project that's representative of large iOS and macOS apps and is a heavy user of macros. Looking at the two Foundation macros mentioned above, `NS_ASSUME_NONNULL_BEGIN` and `NS_AVAILABLE`, they are together used 352 times in WebKit's code base.

Problems with the lack of macros in the Modules TS

The [Modules TS](#) lacks support for macros. According to Section 3.2 in [p0142r0](#):

... because the preprocessor is largely independent of the core language, it is impossible for a tool to understand (even grammatically) source code in header files without knowing the set of macros and configurations that a source file including the header file will activate. It is regrettably far too easy and far too common to under-appreciate how much macros are (and have been) stifling development of semantics-aware programming tools and how much of drag they constitute for C++, compared to alternatives...

While modules may seem like an attractive way to obsolete macros, the reality is that Apple's platforms depend on macros. Developers on our platform will not benefit from modules unless they work well with macros. Additionally, concerns from others were already outlined in [P0273R1](#) and [P0837R0](#).

We propose support for macros with the intent of helping with migration. To achieve it, we suggest adding extra syntax that:

- Adds on top of existing [Modules TS](#) syntax.
- Can easily be later deprecated and removed, without polluting the reserved keyword namespace.

Proposed macro syntax

To export macros defined in a module, we propose augmenting the *module-declaration* in a *module interface unit* with a special suffix naming:

```
export module M; // declare module M
...

#define INFINITY ...
#define HUGE_VAL ...

...

export M.*; // M export INFINITY, HUGE_VAL, etc
```

In the code snippet above, all macros in `M`'s *module interface unit* are exported. To select macros to export, a macro identifier is specified with `export M.<MACRO_NAME>`. As illustrated in the example above, globbing is also supported, making the task of exporting groups of macros handy.

```
export M.#INFINITY; // M exports macro INFINITY
...
export M.#HUGE_*; // M exports macro HUGE_VAL
```

On the module consumer side, no fine grained approach is available and one can only import the complete set of macros exported by module `M`:

```
import M; // import M with macros exported in module M (considering M exported any)
```

Exporting macros in module `M` are the only way to control what macros will show up in the importer side, that's where the judicious use of macros should be controlled.

It's also important to note that the dotted module names in the Modules TS don't indicate a module-submodule relationship or filename hierarchy, which also has been the subject of other papers (see [P0778R0](#)). However, we propose that the suffix `.#` has special meaning, regardless of the amount of dots prior to the end of the module name.

The use of `export t M. #<MACRO_NAME>` is only valid if the macro is visible at the point of export. Same applies after glob expansion, only the visible ones are selected.

Representing macros by wrapping them up under a specific suffix naming provides the necessary syntactic sugar that allows for deprecation of the mechanism later on; there's no pollution of the top-level reserved keywords.

Rules for macro definitions

Different modules can have opposing definitions for the same macro, for example, one module might `#define` a macro while the other `#undef` it. We need a model with some rules on how it should behave. This paper proposes to reuse a mechanism model similar to the one defined in the [Clang Modules documentation](#). The relevant rules extracted from the document follow:

- Each `#define` and `#undef` of a macro is considered to be a distinct entity.
- A `#define X` or `#undef X` directive **overrides** all definitions of X that are visible at the point of the directive.
- A `#define` or `#undef` directive is **active** if it is visible and no visible directive overrides it.
- A set of macro directives is consistent if it consists of only `#undef` directives, or if all `#define` directives in the set define the macro name to the same sequence of tokens (following the usual rules for macro redefinitions).
- If a macro name is used and the set of active directives is not consistent, the program is ill-formed. Otherwise, the (unique) meaning of the macro name is used.

Example A

Also extracted from that document, suppose this example:

- `<stdio.h>` defines a macro `getc` (and exports its `#define`)
- `<cstdio>` imports the `<stdio.h>` module and undefines the macro (and exports its `#undef`)

The `#undef` **overrides** the `#define` , and a source file that imports both modules in any order will not see `getc` defined as a macro.

Example B

Also suppose a module `M` exporting macros `FOO` and `BAR` :

```
export module M;

#define FOO puts("hello")
#define BAR FOO
#undef FOO
#define FOO puts("world")
...

export M.*;
```

Both `FOO` and `BAR` macros contain `puts("world")` as a result of importing; `BAR` will contain whatever `FOO` contains at the end of module, since both are **active** at that point.

X-Macros

The proposed model, as the paper title suggests, is intended to support *modular macros*, meaning that this paper has no intent to support idioms that are intrinsically *non-modular*.

[X-Macros](#) is a popular non-modular idiom where a macro is defined and subsequently expanded by a `#include`. For instance, take a look at LLVM's `include/llvm/BinaryFormat/Dwarf.h`:

```
...
enum LineNumberOps : uint8_t {
#define HANDLE_DW_LNS(ID, NAME) DW_LNS_##NAME = ID,
#include "llvm/BinaryFormat/Dwarf.def"
};
```

where `include/llvm/BinaryFormat/Dwarf.def` contains:

```
...
// Line Number Standard Opcode Encodings.
HANDLE_DW_LNS(0x00, extended_op)
HANDLE_DW_LNS(0x01, copy)
HANDLE_DW_LNS(0x02, advance_pc)
...
#undef HANDLE_DW_LNS
```

The expansion for `HANDLE_DW_LNS` in `Dwarf.def` is highly dependent on the context and macro definition in `Dwarf.h`. Such idioms must continue to rely on plain `#include`s.

Acknowledgments

Thanks to Vassil Vassilev, JF Bastien, Adrian Prantl, Duncan P. Exon Smith and Richard Smith for comments and reviews.